

Lizard Bot - Software Specification Document

Version 1.0 - Node.js Single Server Architecture

12. Change Log

Version	Date	Author	Description
1.0	2025-09-15	David Caldwell	Initial release of the Software Specification document.

1. Executive Summary

1.1 Project Overview

The Lizard Bot is a Discord application designed to monitor text channels and automatically respond with "lizard" when the keyword "lizard" is detected in messages. This document specifies the technical implementation using a single Node.js server architecture optimized for cost-effectiveness and reliability.

1.2 Key Architectural Decisions

- **Single Node.js Application:** Monolithic architecture with modular internal design
- **AWS t4g.nano Instance:** ARM-based EC2 instance for optimal price/performance
- **Local SQLite Storage:** Primary data storage with cloud backup to S3
- **Persistent WebSocket:** Direct connection to Discord Gateway for real-time events
- **PM2 Process Management:** Production-grade process monitoring and auto-restart

1.3 Cost Optimization

- **Monthly Operating Cost:** \$3.20/month (AWS t4g.nano reserved instance)

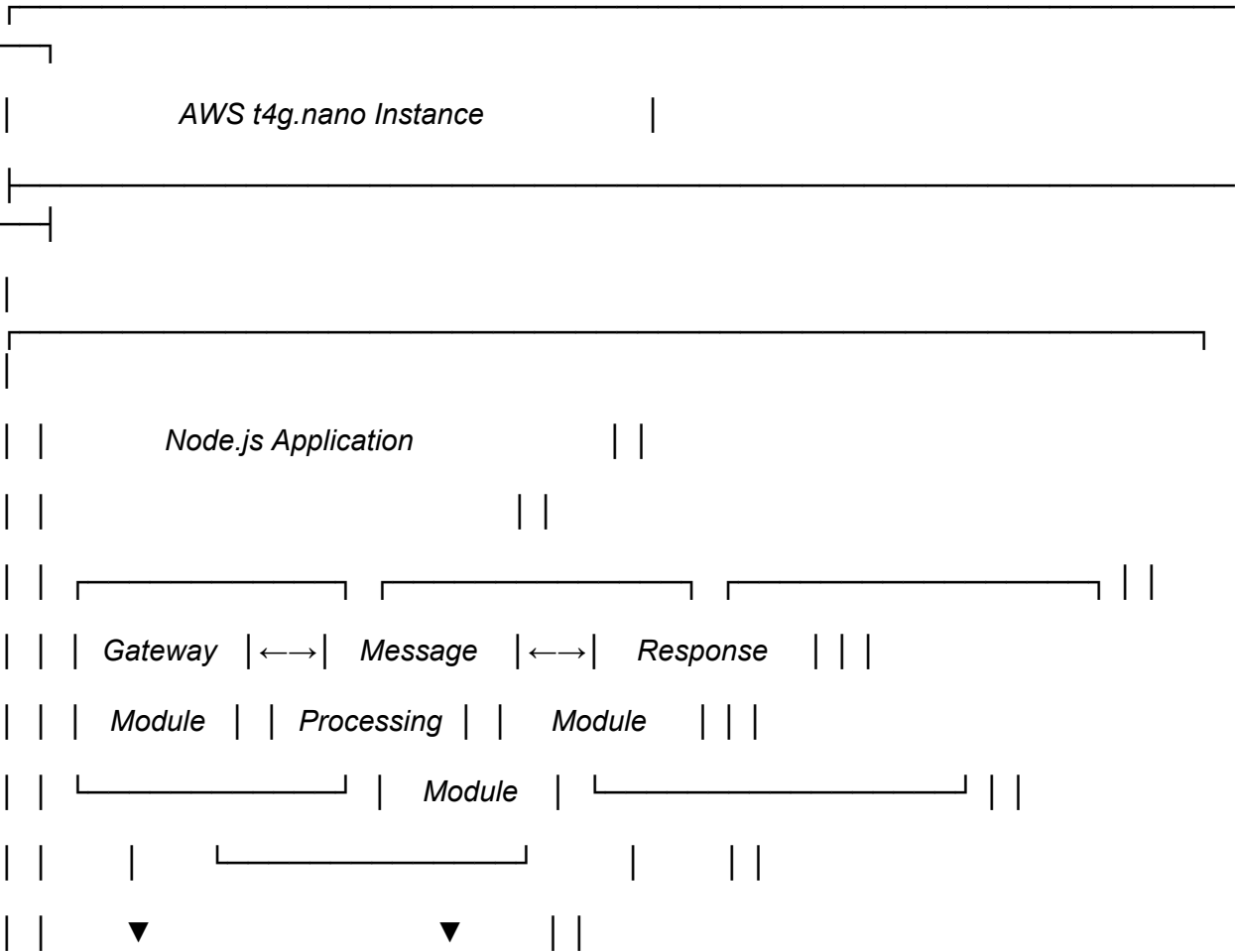
- **Alternative Free Option:** Oracle Cloud ARM Ampere (Always Free tier)
- **Storage Costs:** Minimal S3 backup costs (<\$0.50/month)
- **Total Solution:** Under \$5/month for production deployment

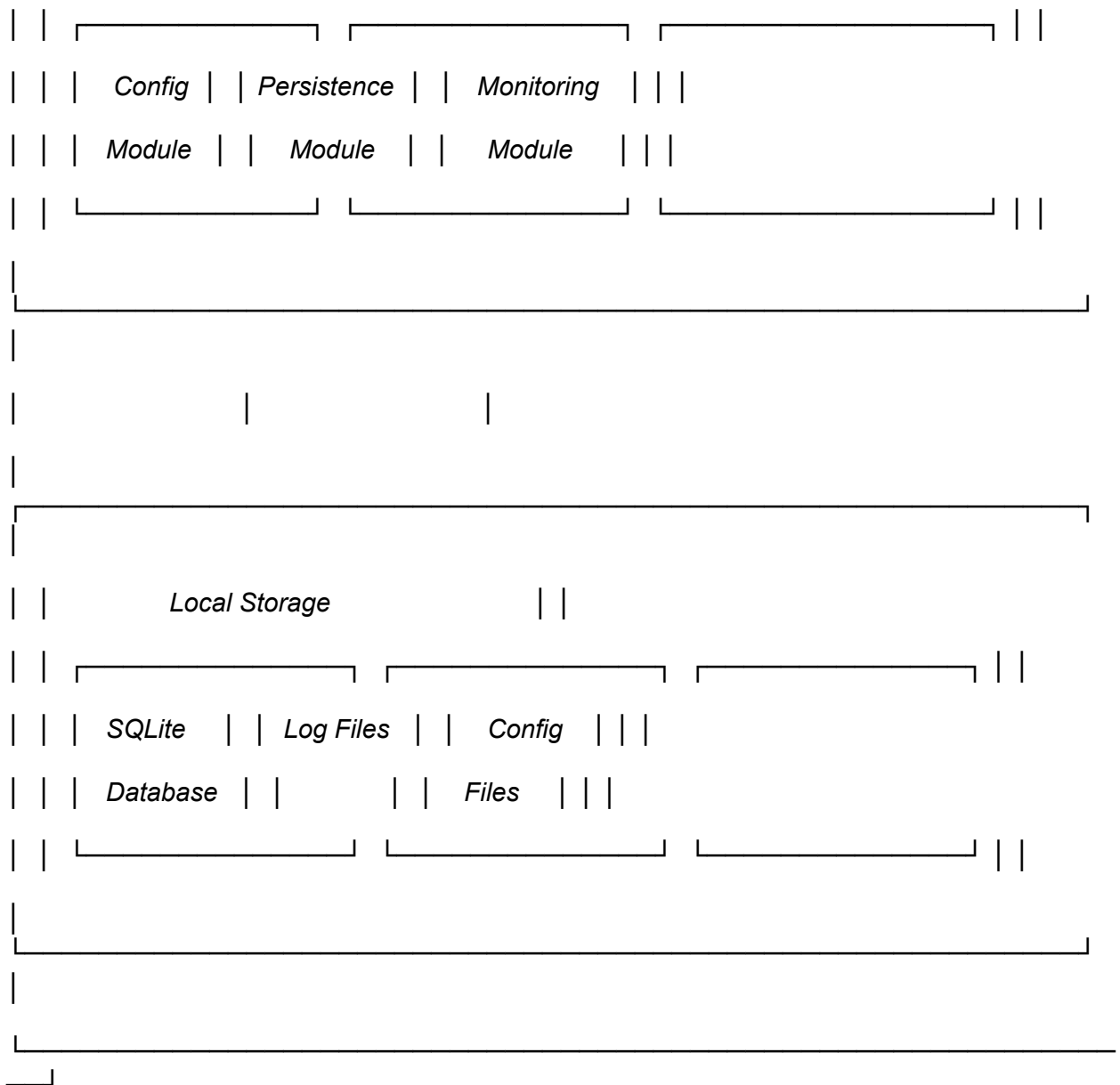
1.4 Technology Stack

- **Runtime:** Node.js 18.x LTS
- **Framework:** Discord.js v14.x
- **Database:** SQLite with S3 backup
- **Process Manager:** PM2
- **Monitoring:** Basic CloudWatch
- **Platform:** AWS EC2 (ARM64)

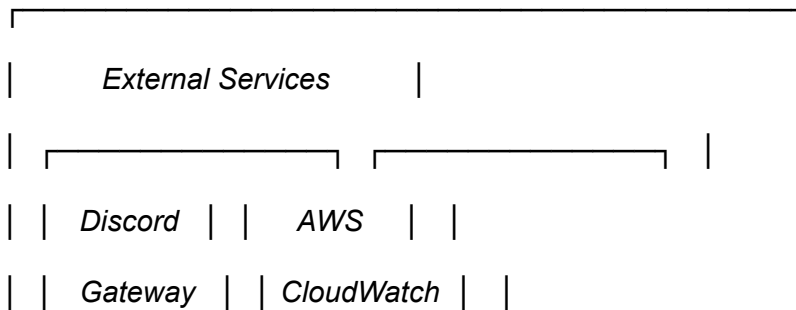
2. System Architecture Overview

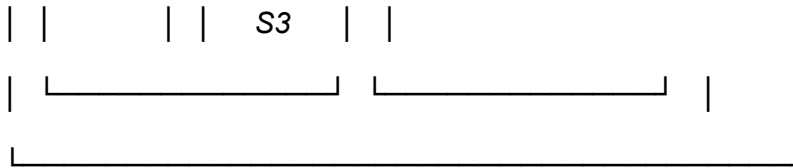
2.1 High-Level Architecture





|





2.2 Application Architecture

The application follows a **modular monolith** pattern with six core modules:

1. **Gateway Module:** Manages Discord WebSocket connection
 2. **Message Processing Module:** Handles keyword detection and business logic
 3. **Response Module:** Manages Discord API responses and rate limiting
 4. **Configuration Module:** Handles runtime configuration and hot-reload
 5. **Persistence Module:** Manages local database and cloud backup
 6. **Monitoring Module:** Provides basic health metrics and alerting
-

3. Application Components

3.1 Discord Gateway Module

Purpose: Maintain persistent WebSocket connection to Discord and handle incoming events.

Key Responsibilities:

- Establish and maintain WebSocket connection to Discord Gateway
- Handle authentication and heartbeat management
- Parse incoming Discord events and route to appropriate handlers
- Implement automatic reconnection with exponential backoff
- Monitor connection health and report status

Configuration Parameters:

- Discord bot token (environment variable)
- Gateway intents configuration
- Reconnection settings (retry count, backoff intervals)
- Connection timeout settings

Error Handling:

- Automatic reconnection on connection loss

- Graceful degradation during Discord maintenance
- Rate limit handling for gateway connections
- Logging of connection events and errors

3.2 Message Processing Module

Purpose: Process incoming Discord messages and determine if responses are needed.

Key Responsibilities:

- Receive message events from Gateway Module
- Perform keyword detection and matching
- Apply business logic rules
- Generate response content
- Rate limiting and cooldown management
- Emit response events

Keyword Detection Engine:

- Case-insensitive matching using regex: `/\blizard\b/i`
- Word boundary detection (matches "lizard" but not "lizards")
- Punctuation handling (works with "lizard!", "lizard.", etc.)
- Configurable keyword list stored in SQLite
- Future: Regular expression support

Cooldown Management:

- Per-channel cooldown tracking (3 seconds default)
- In-memory Map structure with automatic cleanup
- Guild-specific cooldown settings (future enhancement)
- Optional persistence across application restarts

3.3 Response Module

Purpose: Handle outgoing Discord messages and API interactions.

Key Responsibilities:

- Receive response events from Message Processing Module
- Send messages to Discord via REST API
- Handle Discord API rate limits
- Implement retry logic for failed requests
- Track basic response metrics

Rate Limiting Strategy:

- *Discord API rate limit compliance (50 requests/second)*
- *Per-guild rate limit tracking*
- *Exponential backoff for rate limit violations*
- *Simple queue management for burst scenarios*

Error Handling:

- *Network error retry logic (3 attempts)*
- *Discord API error handling*
- *Graceful degradation for service unavailability*
- *Basic error logging*

3.4 Configuration Module

Purpose: *Manage application configuration and provide hot-reload capability.*

Key Responsibilities:

- *Load configuration from files and environment variables*
- *Provide configuration to other modules*
- *Support hot-reload of configuration changes*
- *Validate configuration parameters*
- *Handle environment-specific settings*

Configuration Sources:

- *Environment variables (production secrets)*
- *JSON configuration files (application settings)*
- *SQLite database (runtime settings)*

Hot-Reload Capability:

- *File system watching for config files*
- *Database polling for runtime settings (every 30 seconds)*
- *Configuration validation before applying*
- *Graceful fallback to previous configuration on errors*

3.5 Persistence Module

Purpose: *Manage local data storage and cloud backup operations.*

Key Responsibilities:

- *SQLite database management*
- *Data schema creation and migration*
- *CRUD operations for configuration and analytics*

- *Automated backup to S3 (daily)*
- *Basic data recovery procedures*

Interface Methods:

- `initializeDatabase()` - Create tables and indexes
- `getGuildConfig(guildId)` - Retrieve guild configuration
- `updateGuildConfig(guildId, config)` - Update guild settings
- `logAnalytics(event)` - Record usage metrics
- `backupToS3()` - Upload database backup

3.6 Monitoring Module

Purpose: Provide basic application health monitoring and alerting.

Key Responsibilities:

- *Simple health check endpoint*
- *Basic metrics collection (message count, response count)*
- *Resource monitoring (memory usage)*
- *Integration with PM2 monitoring*
- *CloudWatch Logs forwarding*

Health Check Endpoint:

```
app.get('/health', (req, res) => {

  res.json({

    status: discordClient.isReady() ? 'healthy' : 'unhealthy',

    uptime: process.uptime(),

    memory: Math.round(process.memoryUsage().heapUsed / 1024 / 1024) + 'MB'

  });

});
```

Basic Metrics Collection:

- *Messages processed count*
- *Responses sent count*
- *Application uptime*

- *Memory usage tracking*
-

4. Data Architecture

4.1 SQLite Database Design

Guild Configuration Table:

```
CREATE TABLE guild_config (  
    guild_id TEXT PRIMARY KEY,  
    guild_name TEXT,  
    keywords TEXT,          -- JSON array: ["lizard"]  
    response_templates TEXT, -- JSON array: ["lizard"]  
    cooldown_seconds INTEGER DEFAULT 3,  
    enabled BOOLEAN DEFAULT 1,  
    settings TEXT,          -- JSON configuration object  
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP,  
    updated_at DATETIME DEFAULT CURRENT_TIMESTAMP  
);
```

Analytics Table:

```
CREATE TABLE analytics (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    guild_id TEXT,  
    channel_id TEXT,  
    keyword_matched TEXT,  
    response_sent BOOLEAN,
```

```
timestamp DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

Application Logs Table:

```
CREATE TABLE app_logs (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    level TEXT,          -- ERROR, WARN, INFO, DEBUG
    module TEXT,        -- Gateway, Processing, Response, etc.
    message TEXT,
    timestamp DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

4.2 In-Memory Data Structures

Cooldown Tracking:

```
// In-memory Map for performance
const cooldowns = new Map(); // channelId -> lastResponseTime
```

Configuration Cache:

```
// Cached guild configurations
const guildConfigs = new Map(); // guildId -> config object
```

4.3 Backup Strategy

Daily S3 Backup:

- Automated SQLite backup to S3 bucket
- 30-day retention policy

- *Simple restore procedure from latest backup*

Backup Command:

Copy database to S3

```
aws s3 cp /opt/lizard-bot/data/lizard-bot.db \
```

```
s3://lizard-bot-backups/daily/lizard-bot-$(date +%Y%m%d).db
```

5. Infrastructure Specifications

5.1 Server Requirements

AWS t4g.nano Specifications:

- **CPU:** 2 vCPU (ARM-based Graviton2)
- **Memory:** 512MB RAM
- **Storage:** 20GB EBS gp3 volume
- **Network:** Up to 5 Gbps burst
- **Cost:** \$3.20/month (3-year reserved)

Operating System:

- **Base OS:** Amazon Linux 2 ARM64
- **Node.js:** Version 18.x LTS
- **Package Manager:** npm
- **Process Manager:** PM2

Network Configuration:

- **VPC:** Dedicated VPC with public subnet
- **Security Group:** SSH (22), Health Check (3000), HTTPS outbound (443)
- **Elastic IP:** Optional for static IP address

5.2 Alternative: Oracle Cloud Free Tier

Oracle ARM Ampere Specifications:

- **CPU:** 4 vCPU (ARM-based)
- **Memory:** 24GB RAM
- **Storage:** 200GB Block Storage

- **Network:** 10TB egress/month
- **Cost:** \$0/month (Always Free)

Trade-offs:

-  Significantly more resources for free
-  No monthly costs
-  Instance availability challenges
-  Potential account termination risk

5.3 Environment Configuration

Development Environment:

- Local Node.js setup or Docker container
- Local SQLite database
- Console logging only
- Mock Discord client for testing

Production Environment:

- EC2 t4g.nano with reserved pricing
 - PM2 process management
 - CloudWatch Logs integration
 - Automated S3 backup
-

6. Security & Compliance

6.1 Application Security

Bot Token Management:

- Store Discord bot token in environment variables
- Use AWS Systems Manager Parameter Store for production
- Never log or expose token in application code

Input Validation:

- Sanitize Discord message content before processing
- Validate message length limits (Discord 2000 character limit)
- Reject messages from bot accounts to prevent loops

Rate Limiting:

- *Implement Discord API rate limiting (50 requests/second)*
- *Per-guild request tracking*
- *Exponential backoff on rate limit violations*

6.2 Infrastructure Security

EC2 Security Group:

Inbound Rules:

- *Port 22 (SSH): VPC only or specific IP ranges*
- *Port 3000 (Health): Load balancer only*

Outbound Rules:

- *Port 443 (HTTPS): 0.0.0.0/0 (Discord API)*
- *Port 80 (HTTP): 0.0.0.0/0 (Package updates)*

IAM Role Permissions:

- *S3 backup bucket access (GetObject, PutObject)*
- *CloudWatch Logs write access*
- *Systems Manager Parameter Store read access (bot token)*

Basic Server Hardening:

- *Automatic security updates enabled*
- *Unnecessary services disabled*
- *SSH key-based authentication only*
- *Regular OS patching schedule*

6.3 Discord API Compliance

Terms of Service Requirements:

- *Bot clearly identifies as automated*
- *No spam or excessive messaging*
- *Respect Discord rate limits*
- *Minimal data collection (guild ID, channel ID only)*
- *No user data storage beyond processing needs*

Permission Management:

- Request minimal required permissions:
 - View Channels
 - Send Messages
 - Read Message History
 - Validate permissions before attempting operations
-

7. Monitoring & Operations

7.1 Basic Monitoring

Health Check System:

- HTTP endpoint at `/health` port 3000
- Returns Discord connection status and basic metrics
- Used by load balancer for health verification

PM2 Monitoring:

- Built-in process monitoring: `pm2 monit`
- Automatic restart on crashes
- Memory usage alerts
- Log file management

CloudWatch Integration:

- Forward application logs to CloudWatch Logs
- Basic EC2 instance metrics (CPU, memory, disk)
- Simple alarm for instance health

7.2 Basic Alerting

Critical Alerts:

- Application process stopped (PM2 monitoring)
- High memory usage (>400MB sustained)
- Discord connection loss (>5 minutes)
- Disk space usage (>80%)

Alert Delivery:

- AWS SNS email notifications
- Basic CloudWatch alarms
- PM2 restart notifications

7.3 Log Management

Application Logging:

- Winston logging library
- Log levels: *ERROR*, *WARN*, *INFO*, *DEBUG*
- Local log files with rotation (7 days retention)
- CloudWatch Logs forwarding for production

Log Structure:

```
logger.info('Message processed', {  
  
  guildId: message.guild.id,  
  
  channelId: message.channel.id,  
  
  keyword: 'lizard',  
  
  responseTime: Date.now() - startTime  
});
```

8. Deployment & Operations

8.1 Simple Deployment Process

Initial Setup:

1. Install dependencies

```
sudo yum update -y
```

```
curl -fsSL https://rpm.nodesource.com/setup_18.x | sudo bash -
```

```
sudo yum install -y nodejs git
```

```
sudo npm install -g pm2
```

2. Deploy application

```
git clone <repository-url> /opt/lizard-bot
```

```
cd /opt/lizard-bot
```

```
npm ci --production
```

3. Configure environment

```
echo "DISCORD_BOT_TOKEN=your_token_here" > .env
```

```
echo "NODE_ENV=production" >> .env
```

4. Start application

```
pm2 start ecosystem.config.js
```

```
pm2 save
```

```
pm2 startup
```

PM2 Configuration:

```
// ecosystem.config.js
```

```
module.exports = {
```

```
  apps: [{
```

```
    name: 'lizard-bot',
```

```
    script: 'index.js',
```

```
    instances: 1,
```

```
    autorestart: true,
```

```
    max_memory_restart: '400M',
```

```
    env: {
```

```
      NODE_ENV: 'production'
```

```
}  
}  
};
```

8.2 Update Procedure

Simple Update Process:

1. Pull latest code

```
cd /opt/lizard-bot
```

```
git pull origin main
```

2. Update dependencies

```
npm ci --production
```

3. Restart application

```
pm2 reload lizard-bot
```

4. Verify health

```
curl http://localhost:3000/health
```

8.3 Backup & Recovery

Daily Backup Script:

```
#!/bin/bash
```

Simple backup to S3

```
DATE=$(date +%Y%m%d)
```

```
cp /opt/lizard-bot/data/lizard-bot.db /tmp/lizard-bot-$(date +%Y%m%d).db
aws s3 cp /tmp/lizard-bot-$(date +%Y%m%d).db s3://lizard-bot-backups/daily/
rm /tmp/lizard-bot-$(date +%Y%m%d).db
```

Recovery Process:

1. Stop application

```
pm2 stop lizard-bot
```

2. Restore database

```
aws s3 cp s3://lizard-bot-backups/daily/lizard-bot-$(date +%Y%m%d).db \
/opt/lizard-bot/data/lizard-bot.db
```

3. Restart application

```
pm2 start lizard-bot
```

9. Performance & Scalability

9.1 Performance Targets

Response Time Requirements:

- **Message Processing:** <200ms from Discord event to response decision
- **Database Queries:** <5ms for configuration lookups
- **Discord API Calls:** <500ms for message sending
- **Health Checks:** <100ms response time

Throughput Specifications:

- **Target:** 200-500 messages per second (single instance)
- **Guilds:** Up to 100 Discord servers simultaneously

- **Concurrent Connections:** Single WebSocket connection handles all guilds

Resource Utilization:

- **Memory:** <400MB RAM (80% of available)
- **CPU:** <50% average usage
- **Disk:** <16GB usage (80% of 20GB)

9.2 Scaling Strategy

Vertical Scaling Options:

- **t4g.micro:** 2 vCPU, 1GB RAM (+100% memory)
- **t4g.small:** 2 vCPU, 2GB RAM (+400% memory)
- **t4g.medium:** 2 vCPU, 4GB RAM (+800% memory)

Performance Optimization:

- SQLite WAL mode for better concurrency
- In-memory caching for frequently accessed data
- Connection pooling for database operations
- Efficient memory management with periodic cleanup

Future Horizontal Scaling:

- Load balancing across multiple instances
- Guild sharding for distribution
- Shared database for coordination
- Message queue for event processing

10. Future Enhancements

10.1 Phase 2 Features (6-month roadmap)

Web Dashboard:

- Express.js web interface for bot configuration
- Guild management and statistics
- Real-time monitoring dashboard
- User authentication for server administrators

Enhanced Commands:

- *Discord slash commands for bot interaction*
- *Custom keyword and response configuration*
- *Per-channel bot enable/disable controls*
- *Usage statistics and analytics*

Advanced Configuration:

- *Multiple keyword support per guild*
- *Custom response templates with variables*
- *Advanced cooldown settings (per-user, per-channel)*
- *Regex pattern matching for keywords*

10.2 Phase 3 Features (12-month roadmap)

Voice Channel Support:

- *Text-to-speech responses in voice channels*
- *Voice keyword detection (future Discord API)*
- *Audio file responses*

Advanced Features:

- *AI-powered response generation*
- *Multi-language keyword detection*
- *Custom bot personalities per guild*
- *Integration with external APIs*

Migration to Microservices:

- *Containerization with Docker*
- *Kubernetes deployment*
- *Service mesh architecture*
- *Horizontal auto-scaling*

11. Resources & References

11.1 Source Documentation

Project Foundation:

- **Project Proposal:** *Initial scope and objectives*
- **Software Requirements Document (SRD):** *Complete functional and non-functional requirements*

- **Requirements Tracking Matrix:** SRD requirement coverage verification

11.2 Technical References

Core Technologies:

- [Discord.js v14 Documentation](#)
- [Discord API Documentation](#)
- [Node.js Best Practices](#)
- [PM2 Documentation](#)
- [SQLite Documentation](#)

AWS Infrastructure:

- [EC2 t4g Instance Types](#)
- [CloudWatch Documentation](#)
- [S3 Documentation](#)

Security & Compliance:

- [Discord Developer Terms of Service](#)
- [Node.js Security Best Practices](#)

This streamlined Software Specification Document provides the essential technical blueprint for implementing the Lizard Bot while maintaining full SRD compliance. The specification focuses on practical implementation needs without over-engineering for a simple Discord bot application.